

Estruturação

FIRE QUEST

Sistema de Gamificação de Tarefas e Gestão de Vida

Resumo do Projeto

O FireQuest é um sistema (aplicativo e website) voltado para a organização da vida acadêmica e pessoal, utilizando conceitos de gamificação inspirados em RPG. A proposta é transformar tarefas, estudos e atividades físicas em missões, incentivando a produtividade, disciplina e evolução contínua do usuário.

A plataforma centraliza informações importantes como anotações, provas, atividades e treinos, promovendo também a colaboração entre usuários e o acompanhamento do desempenho individual.

Objetivo

Criar um ambiente que una:

- organização acadêmica
- desenvolvimento pessoal
- engajamento por gamificação

Permitindo que o usuário evolua de forma mensurável em diferentes áreas da vida.

Estrutura do Sistema

Missões (Tarefas)

Cada atividade cadastrada no sistema é tratada como uma missão e possui dois parâmetros principais:

- **Dificuldade (0 a 10):** nível de esforço necessário
- **Urgência (0 a 10):** prioridade de execução

Esses valores são definidos por votação entre os usuários.

Sistema de XP

A pontuação é calculada de forma simples e transparente:

XP base:

$$\text{XP} = \text{Dificuldade} + \text{Urgência}$$

Exemplo:

- Dificuldade: 10
 - Urgência: 4
→ XP = 14
-



Penalidade por Atraso

- Tarefas entregues fora do prazo recebem:

-50% da pontuação total



Sistema de Validação

Toda missão precisa ser validada pela comunidade.

A recompensa final é proporcional ao nível de aprovação:

- 100% aprovação → 100% do XP
- 50% aprovação → 50% do XP
- aprovação parcial → XP proporcional

Esse sistema garante:

- justiça
 - transparência
 - engajamento coletivo
-



Sistema de Habilidades (Skills)

As atividades são organizadas em categorias que representam áreas de desenvolvimento:



Acadêmicas

- Exatas
- Programação
- Humanas



Físicas

- Flexão
- Abdominal
- Prancha
- Barra

Cada atividade contribui diretamente para a evolução da respectiva habilidade.



Acompanhamento de Progresso

O sistema contará com:

- barra de XP (visualização contínua do progresso)
 - gráficos de desempenho por habilidade
 - histórico de atividades
-



Sistema de Ranking

O ranking será baseado principalmente no:

XP total acumulado

Também haverá formas adicionais de visualização:

- ranking entre amigos
 - ranking por período (ex: semanal)
 - ranking por categoria
-



Sistema de Pontos e Recompensas

- **1 XP = 1 ponto (pt)**

Os pontos podem ser utilizados para “comprar” momentos de lazer, como:

- assistir filmes
- jogar
- descansar

A validação dessas recompensas é opcional e definida pelo próprio usuário, promovendo autonomia e equilíbrio entre produtividade e descanso.



Semanais

- Definidas pelo usuário
- Relacionadas a estudo, tarefas ou exercícios



Diária fixa

- Beber pelo menos **2 litros de água**



Diferencial do Projeto

O FireQuest não é apenas um organizador de tarefas, mas um sistema completo que integra:

- produtividade
- saúde física
- colaboração social
- gamificação

Transformando a rotina em uma experiência progressiva, motivadora e mensurável.

Link do Repositório no Github:

➤ [GitHub - FireQuest](#)

Documentação



DOCUMENTAÇÃO COMPLETA — FIREQUEST BACKEND



PARTE 1 — VISÃO GERAL + ARQUITETURA



1. VISÃO GERAL DO SISTEMA

O **FireQuest** é uma API backend que implementa um sistema completo de:

- gestão de tarefas (missões)
 - gamificação (XP, ranking)
 - metas pessoais
 - histórico e analytics
 - autenticação segura
-



PROPÓSITO

Permitir que um usuário:

- crie tarefas (missões)
 - conclua tarefas
 - ganhe XP
 - acompanhe progresso
 - compare com outros usuários
 - crie metas pessoais
-



COMO O SISTEMA FUNCIONA

O backend é uma API REST que:

- recebe requisições (HTTP)
 - processa regras de negócio
 - retorna dados em JSON
-



FLUXO PRINCIPAL

Usuário → Login → Recebe Token → Faz ações autenticadas

Exemplo:

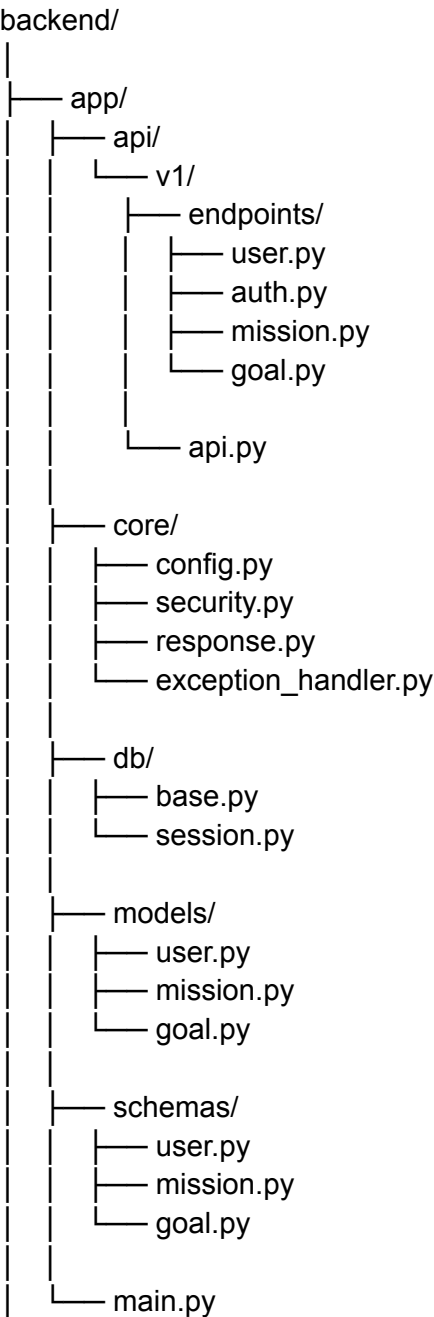
Login → Criar missão → Concluir missão → Ganhar XP → Ranking



2. ARQUITETURA DO PROJETO



Estrutura do backend



COMPONENTES

Camada Responsabilidade

e

models	estrutura do banco
schemas	validação de dados
endpoints	lógica da API
core	segurança, config
db	conexão banco

3. AUTENTICAÇÃO (MUITO IMPORTANTE)

COMO FUNCIONA

O sistema usa:

👉 JWT (JSON Web Token)

Fluxo

1. Login

POST /api/v1/auth/login

Envia:

username + password

Recebe:

access_token

2. Uso do token

Todas as rotas protegidas exigem:

Authorization: Bearer TOKEN

IMPORTANTE PARA FRONTEND

👉 TODA requisição (exceto login e cadastro) deve enviar token

4. USUÁRIOS

Criar usuário

POST /api/v1/users/

Login

POST /api/v1/auth/login

Ver usuário logado

GET /api/v1/users/me

Ranking

GET /api/v1/users/ranking

Estrutura do usuário

```
{
  "id": 1,
  "username": "gui",
  "email": "...",
  "xp": 100,
  "streak": 0
}
```

5. MISSÕES (CORE DO SISTEMA)

Criar missão

POST /api/v1/missions/

Campos:

```
{
  "title": "...",
  "description": "...",
  "difficulty": 0-10,
  "urgency": 0-10,
  "due_date": "opcional"
}
```

REGRA DE XP

XP = dificuldade + urgência

Listar

GET /missions/
GET /missions/pending
GET /missions/completed

Concluir missão

PUT /missions/{id}/complete

REGRAS DE XP AVANÇADAS

Atraso

$XP = XP * 0.5$

Validação

$XP = XP * (\% \text{ aprovação} / 100)$

6. METAS

Criar meta

POST /api/v1/goals/

Campos:

```
{  
  "title": "...",  
  "description": "...",  
  "is_daily": true/false  
}
```

7. HISTÓRICO E STATS

Histórico

GET /missions/history

Estatísticas

GET /missions/stats

Retorna:

{

```
"total_missions": 10,  
"completed": 7,  
"pending": 3,  
"xp_total": 120  
}
```



8. PADRÃO DE RESPOSTA



SUCESSO

```
{  
  "success": true,  
  "data": {...}  
}
```



ERRO

```
{  
  "success": false,  
  "error": "mensagem"  
}
```



9. REGRAS IMPORTANTES PARA FRONTEND



1. SEM TOKEN = SEM ACESSO



2. SEMPRE USAR JSON



3. SEMPRE TRATAR SUCCESS



4. NUNCA CONFIAR NO FRONTEND

Toda validação está no backend.



10. O QUE O FRONTEND PRECISA FAZER

WEB / MOBILE DEV DEVE IMPLEMENTAR:



AUTENTICAÇÃO

- tela login
 - guardar token
-



MISSÕES

- criar missão
 - listar
 - concluir
-



DASHBOARD

- XP total
 - ranking
 - estatísticas
-



METAS

- criar
- marcar



DOCUMENTAÇÃO — FIREQUEST



PARTE 2 — FRONTEND (WEB + MOBILE)



1. VISÃO DO FRONTEND

O frontend (web + mobile) será responsável por:

- interface do usuário

- interação com a API
- exibição de dados
- experiência visual (HUD futurista)



PRINCÍPIO FUNDAMENTAL

👉 O frontend **NÃO** tem lógica de negócio

Tudo vem do backend.



FLUXO GLOBAL

Usuário abre app



Login



Recebe token



Armazena token



Usa token em TODAS requisições



Interage com sistema



2. TECNOLOGIAS



WEB

- React
- Axios (requisições)
- CSS (ou Tailwind)



MOBILE

- React Native
 - Expo (opcional)
 - Axios
-



3. ESTRUTURA DO FRONTEND



WEB (/frontend)

```
src/
├── api/
│   └── api.js
├── pages/
│   ├── Login.jsx
│   ├── Register.jsx
│   ├── Dashboard.jsx
│   ├── Missions.jsx
│   ├── Goals.jsx
│   └── Ranking.jsx
├── components/
│   ├── Navbar.jsx
│   ├── MissionCard.jsx
│   ├── GoalCard.jsx
│   └── XPBar.jsx
├── context/
│   └── AuthContext.jsx
└── App.jsx
```



MOBILE (/mobile)

```
src/
├── screens/
│   ├── LoginScreen.js
│   ├── DashboardScreen.js
│   ├── MissionsScreen.js
│   └── GoalsScreen.js
├── components/
├── api/
└── context/
```



4. AUTENTICAÇÃO (PASSO CRÍTICO)



COMO FUNCIONA

1. Login

POST /auth/login



Código (WEB)



api/api.js

```
import axios from "axios";

const api = axios.create({
  baseURL: "http://127.0.0.1:8000/api/v1"
});

export default api;
```



Login

```
import api from "../api/api";

async function login(username, password) {
  const response = await api.post("/auth/login", {
    username,
    password
  });

  return response.data.access_token;
}
```



SALVAR TOKEN

```
localStorage.setItem("token", token);
```



USAR TOKEN AUTOMATICAMENTE

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");

  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }

  return config;
});
```

5. MISSÕES (TELA PRINCIPAL)

LISTAR MISSÕES

GET /missions/pending

COMPONENTE

MissionCard.jsx

```
export default function MissionCard({ mission, onComplete }) {
  return (
    <div className="card">
      <h3>{mission.title}</h3>
      <p>{mission.description}</p>

      <p>XP: {mission.xp}</p>

      <button onClick={() => onComplete(mission.id)}>
        Concluir
      </button>
    </div>
  );
}
```

CONCLUIR MISSÃO

```
async function completeMission(id) {
  await api.put(`/missions/${id}/complete`);
}
```

6. DASHBOARD

DADOS

GET /missions/stats

EXIBIR

- XP total
- total de missões
- concluídas

- pendentes

XP BAR

XPBar.jsx

```
export default function XPBar({ xp }) {  
  return (  
    <div className="xp-bar">  
      <div style={{ width: `${xp}%` }} className="fill"></div>  
    </div>  
  );  
}
```

7. RANKING

ENDPOINT

GET /users/ranking

UI

```
ranking.map((user, index) => (  
  <div key={user.username}>  
    #{index + 1} - {user.username} ({user.xp} XP)  
  </div>  
));
```

8. METAS

CRIAR META

POST /goals/

UI

- checkbox
 - botão "concluir"
-
-

🎨 9. DESIGN (CRÍTICO)

Agora o mais importante para seu projeto se destacar:

🎨 ESTILO OBRIGATÓRIO

- fundo escuro (preto/roxo)
- azul neon
- ciano
- transparência (glassmorphism)
- glow

🧠 REFERÊNCIA VISUAL

Painéis futuristas (estilo daqueles animes isekai da vida)



📦 CSS BASE

```
body {  
  background: #0a0a0f;  
  color: #00eaff;  
}
```

```
.card {
```

```
background: rgba(0, 0, 0, 0.5);
border: 1px solid #00eaff;
box-shadow: 0 0 10px #00eaff;
border-radius: 10px;
}
```

10. MOBILE (RESUMO)

MESMA LÓGICA

- mesmas rotas
 - mesmo token
 - mesma API
-

DIFERENÇA

- UI adaptada
 - navegação com stack
-
-

11. REGRAS IMPORTANTES

NÃO FAZER

- lógica de XP no frontend
 - validação crítica no frontend
 - guardar senha
-

SEMPRE FAZER

- usar token
 - tratar erro
 - usar `success`
-
-



12. ROADMAP FRONTEND

ETAPA 1

- login ✓
- registro ✓

ETAPA 2

- dashboard ✓
- XP ✓

ETAPA 3

- missões ✓
- criar + concluir ✓

ETAPA 4

- ranking ✓

ETAPA 5

- metas ✓
-



CONCLUSÃO

Seu backend está:

- 👉 100% pronto
- 👉 estruturado
- 👉 escalável

Seu frontend agora precisa:

- 👉 apenas consumir corretamente
-



STATUS

Backend: FINALIZADO ✓

Frontend: PRONTO PARA INICIAR 🚀